

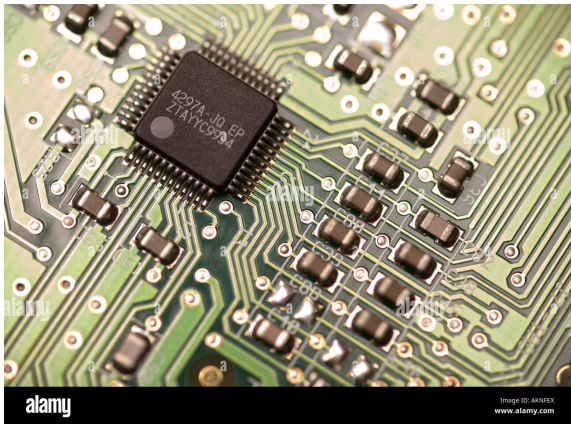
Introdução à Computação

representação de dados no computador

Alexandre Rademaker

representação de dados I

Circuitos eletrônicos transmitem sinais. Mas como representar informação com eletrecidade?



representação de dados II

Com um único fio condutor, podemos indicar dois estados (bits): ligado e desligado. Sim/Não, 0/1, Verdade/Falso etc.

Mais fios, mais bits:



representação binária I

Em um computador, tudo são bits: 0 ou 1

representação binária II

Números decimais

$$123 = 1 * 10^2 + 2 * 10^1 + 3 * 10^0 = 100 + 20 + 3$$

Números binários

$$1101 = 1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0 = 8 + 4 + 0 + 1 = 13$$

representação binária III

binário	decimal
000	0
001	1
010	2
011	3
100	4
...	
111	$1 * 2^2 + 1 * 2^1 + 1 * 2^0 = 7$

Para representar um número maior que 7? Maioria dos computadores usam 'palavras' de 8 bits ($11111111 = 255$), um byte.

Com 32 bits podemos representar 2^{32} (4 bilhões de números).

representação de texto I

Um caractere pode ser representado por um byte.

$$A = 65 = 01000001$$

Um padrão de codificação foi definido, a tabela [ASCII](#).

Uma sequência de caracteres

$$HI! = 010010000100100100100001$$

representação de texto II

Com 8 bits temos 2^8 possíveis valores (0 até 255)

Para outros caracteres como acentos temos **Unicode** que usa mais bits por caractere.

Um **emoji** é apenas um número mapeado na tabela Unicode para uma descrição (diferentes empresas usam diferentes imagens)

“face with medical mask” = 11110000 10011111 10011000
10110111

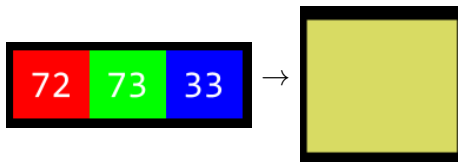
representação de texto III

```
(venv) ar@tenis Temp % cat > teste.txt
Alexandre
^C
(venv) ar@tenis Temp % hexdump -C teste.txt
00000000  41 6c 65 78 61 6e 64 72  65 0a                |Alexandre.|
0000000a
(venv) ar@tenis Temp % file teste.txt
teste.txt: ASCII text
(venv) ar@tenis Temp %
```

... não existem arquivos “texto”!

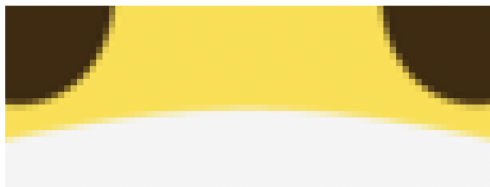
representando imagens I

Usando bits mapeamos números para cores também. Sistema mais usado é o **RGB**



representando imagens II

Os pontos são pixels (3 bits) e um array de pixels compõe uma imagem.



áudios, vídeos e outros I

Vídeos são apenas sequencias de imagens como um flipbook.



áudios, vídeos e outros II

Músicas também representadas como bits, o formato [MIDI](#) com números representando cada notas, duração e volume.

Outros formatos mais sofisticados existem que permitem compressão de dados, combinação de conteúdos diversos (imagens, textos e áudio) dentre outros recursos. Veja [PPM](#), [MP3](#) e [DOCX](#).

Diferentes empresas sugerem formatos para serem usados por seus programas e organizações como a [Unicode Consortium](#), [IEEE](#), [W3C](#) tentam padronizar formatos.

Observação

Uma observação interessante.

1 kilobyte tem $2^{10} = 1.024$ bytes e não $10^3 = 1.000$ bytes.

Lembre-se que computadores operam em binário, contam em potências de 2: 2, 4, 8, 16, 32 etc.

Usamos $2^{10} = 1024$ bytes para um kilobyte porque os computadores operam em base 2 (binário), e potências de 2 são mais fáceis de manipular eletronicamente.

https://en.wikipedia.org/wiki/Binary_prefix